

From Poker to Pokémon: Belief-Based Search for Champions VGC

A white paper on porting the modern poker-AI stack to competitive doubles Pokémon.

ABRA project · SLOWKING track · Reg M-B

Abstract

Two-player competitive Pokémon (VGC) and heads-up poker are, formally, the same *kind* of object: a two-player, zero-sum, imperfect-information game. Over 2007–2021 the poker-AI community produced a nearly complete recipe for solving such games — counterfactual regret minimization (CFR), depth-limited continual re-solving with learned value functions (DeepStack, Libratus), and finally a general self-play RL+search framework over *public belief states* (ReBeL). This paper argues that VGC is best attacked as an imperfect-information game with that toolbox, works through the correspondence term by term, and is honest about the three places the analogy breaks — simultaneity, action-space/horizon scale, and the nature of chance — and what each break forces us to do differently. We describe SLOWKING, an implemented and unit-tested realization of this program, and lay out the remaining path (engine integration, self-play, value-network training) to a genuinely strong agent.

1. Why poker is the right analogy (and Chess/Go are not)

AlphaZero-style search dominates **perfect-information** games (Chess, Go): there is one true state, both players see it, and minimax/MCTS over that state is sound. Naively porting this to Pokémon fails, because a player does **not** know the opponent's team of six is really four they'll bring, their items, abilities, or the fourth move on each set. That hidden information is not noise to be averaged away; it is *strategically exploitable*, which means the solution concept is a **Nash equilibrium in mixed strategies**, not a single best move.

This is exactly poker's situation — you don't see your opponent's cards — and it is why RL+search methods that are sound in Chess **break** in imperfect-information games (ReBeL). The correct mental model for "what should I click" in VGC is "what is my equilibrium strategy given a belief over what my opponent is holding," which is the poker question verbatim.

2. The poker-AI lineage, and what each piece buys us

CFR / MCCFR (Zinkevich et al., 2007). Counterfactual regret minimization is an iterative self-play algorithm whose average strategy provably converges to a Nash equilibrium in two-player zero-sum imperfect-information games (CFR foundations). Regret matching is the atomic update. *For us:* this is the solver at every decision point. Our `nash.py` and `ismcts.py` use regret matching and we have verified they recover exact equilibria (Rock-Paper-Scissors → uniform with zero exploitability; a structured 2×2 → the LP's exact mixed equilibrium).

DeepStack (Moravčík et al., 2017). You cannot run CFR over the entire game tree of no-limit hold'em, so DeepStack introduced **depth-limited continual re-solving**: at each decision, re-solve a

small subgame from the current situation, and where the subgame is cut off, substitute a **deep counterfactual-value network** that predicts the values CFR would have produced ([DeepStack](#)). *For us*: a doubles game is short (median 6, ~10 at the 90th percentile — see §4b) but enormously branchy, so we adopt depth-limited re-solving with a leaf evaluator over an *abstracted* action set. In v1 the leaf value is an engine rollout (MEDICHAM / the real simulator); in v2 it is a trained value network — one that, thanks to the short horizon, is easier to learn than poker's.

Libratus (Brown & Sandholm, 2017). Independently reached continual re-solving via **safe and nested subgame solving** — refining strategy in the subgame actually reached, with guarantees that re-solving does not make you exploitable ([Libratus, Science](#)). *For us*: the "safety" concern maps to re-anchoring the belief correctly each turn so that re-solving a subgame does not leak an exploitable pattern.

ReBeL (Brown et al., 2020). The synthesis. ReBeL redefines the state to be a **public belief state (PBS)** — the public observations plus the probability distribution over every player's private information — and shows that on PBSs an imperfect-information game can be treated like a perfect-information one, so self-play RL+search converges to Nash. It trains a value network and a policy network on PBSs via self-play and reaches superhuman poker with far less domain knowledge than its predecessors ([ReBeL](#)). *For us*: this is the **target** architecture, and the training plan is ReBeL's. **Honesty note**: SLOWKING's *current* search is not yet full ReBeL — it is Information-Set MCTS with **root determinization** (sample a hidden world, search it), which is the well-known PIMC family and suffers **strategy fusion** (it acts as if it will later know hidden info it won't) and non-locality ([Long et al. 2010](#)). ReBeL exists precisely to avoid this by searching over the belief itself. So today's engine is a documented rung *below* ReBeL; the path up is either the $\alpha\mu$ repair of PIMC or true PBS re-solving with a value net (which we have now begun training — see `engine/train_value.py`).

Player of Games (Schmid et al., 2021). Generalizes further, unifying perfect- and imperfect-information search in one algorithm. *For us*: the north star — a single engine spanning team preview and in-battle play.

3. The correspondence, term by term

Poker concept	Pokémon (Champions VGC) analog
Your private hand	Your team's hidden details the opponent hasn't seen (4th moves, spreads, tera/mega intent)
Opponent's private hand	Which 4 of their 6 they bring; their items/abilities/movesets
The board / community cards	The public battle state — mons on field, HP, revealed moves, weather, side conditions
A betting action	A turn's joint choice (moves + targets, switches, Protect)
Chance (the deck)	The engine's stochastic resolution — damage rolls, crits, misses, speed ties
Bluff / balanced range	Mixing your line (e.g., Protect vs. attack) so you cannot be read
Belief over opponent's hand	<code>belief.Belief</code> : distribution over bring (15 subsets) + per-mon set posteriors
Public belief state (ReBeL)	(public board) + (that belief) — SLOWKING's search node
Counterfactual value network	Leaf evaluator of a PBS; v_1 = engine rollout, v_2 = learned net
Continual re-solving	Re-solve a depth-limited subgame each turn, then observe and update belief

4. Where the analogy breaks — and the consequences

The engineering discipline of this project is being precise about the disanalogies, because each one dictates an algorithmic choice.

(a) Simultaneity. Poker is sequential: players act one at a time, and the information sets form an extensive-form tree that sequential CFR is built for. A Pokémon turn is **simultaneous** — both trainers lock in choices without seeing the other's. Vanilla sequential CFR does not directly apply at a turn node. The correct tool is **simultaneous-move regret matching / SM-MCTS**, which is Hannan-consistent and whose average strategy converges to the stage-game Nash ([SM-MCTS analysis](#)). Concretely: at a node we run regret matching *for both players jointly*, using per-action counterfactual values against the opponent's sampled action. Our `ismcts.py` implements exactly this and, as a correctness check, recovers the asymmetric 2×2 equilibrium rather than collapsing to uniform (which a naive single-sample update does — a bug we found and fixed).

**** (b) Action space and horizon — the tree is *wide and shallow*. **** No-limit hold'em has a handful of actions per decision. A doubles turn is the opposite: per active Pokémon, (each legal move $\times$ each legal target) plus switches — easily dozens of joint choices per side, so *hundreds* of joint actions per turn. But the horizon is short: in our ladder data the median game is **6 turns** and the 90th percentile is **10** (mean 6.2, max 64). This is the crucial structural fact, and it inverts the usual poker emphasis. Poker is narrow-and-deep (few actions, many rounds); Champions is **wide-and-shallow** (huge branching, ~ 6 – 10 plies). Consequences: (i) depth-limited search can often see *near the end of the game*, so the leaf value network has a short horizon to cover and is easier to train than DeepStack's; (ii) the binding constraint is **breadth**, not depth — action abstraction (pruning dominated moves, bucketing targets) is where the real work is, more so than the value net. Exact solving is still hopeless on branching alone, so depth-limited search + a leaf evaluator (DeepStack/ReBeL) remains the right frame — just with the effort reallocated from depth to breadth.

(c) The nature of chance — and our unfair advantage. In poker, chance is the deck; the agent must reason about card distributions it cannot observe. In Pokémon, chance is damage variance, critical hits, accuracy, and speed ties — and, crucially, **we possess the exact generator**: the open-source Showdown Champions engine. We do not *model* the dynamics and inherit modelling error (the failure mode that plagued hand-built Pokémon AIs and, frankly, ABRA's own first browser engine); we **call the real simulator** and sample chance exactly. Discovering the engine is open is the finding that makes the whole ReBeL program tractable here — it turns the transition function from "the hardest thing to learn" into "a function call."

(d) A second game on top: team preview. Before the battle, both players simultaneously choose which four of six to bring. This is itself a **one-shot simultaneous matrix game** whose payoffs are win probabilities of one 4-mon bring against another. It is small (15×15) and solvable to an exact equilibrium **mix** — and it exposes the deepest lesson: there is usually **no single "best team you bring."** In a cyclic metagame the equilibrium bring is a *mixture*, because any deterministic bring is exploitable. SLOWKING's `solver.team_preview` returns that mix; this is also the correct reframing of what DITTO *should* be doing (a real meta-game solve), and it is why "give me the one best team" is, strictly, the wrong question.

5. SLOWKING: the implemented synthesis

The `engine/slowking/` package is the program above, built and unit-tested:

- `nash.py` — regret matching + linear-programming equilibrium solving for 2p zero-sum matrices. The atom, verified against known games.
- `belief.py` — the public-belief-state model: a grounded prior over the opponent's bring and sets (from ABRA ladder data) and a Bayesian filter that sharpens it as moves, items, and abilities are revealed. Supplies determinized samples for search.
- `ismcts.py` — Information-Set MCTS with simultaneous-move regret matching and root determinization over the belief. Recovers stage-game equilibria.
- `game.py` — the engine-agnostic game interface; `ChampionsGame` is the adapter to the open Showdown simulator (the one integration still to be wired).
- `solver.py` — the orchestrator: team-preview Nash and in-battle continual re-solving, each returning a **mixed strategy plus its win-probability value** — precisely the object we want.

6. The EV calculation, stated explicitly

Sections 1–5 argue the analogy and name the components. This section writes down the quantity the whole project exists to compute, names each term in poker's vocabulary, and says plainly which terms are finished and which are not. Everything numeric here is measured, and the measurement is named.

6.1 The object

For a joint action `a` (both of my Pokémon's choices) in public state `s`:

$$EV(a) = \sum_r P(r) \cdot \sum_b \sigma_{opp}(b \mid r) \cdot V(T(s, a, b))$$

term	poker name	what it is here	ABRA component	
<code>r</code> , <code>P(r)</code>	range	which 4 of their 6 they brought, and their sets	<code>belief.py</code> , CHOMP	
<code>σ_{opp}(b \</code>	<code>r)</code>	range vs range	the <i>frequencies</i> of their actions, not a guess at one move	XATU
<code>T(s, a, b)</code>	the deck	damage rolls, crits, accuracy, speed ties	<code>champions_sim</code>	
<code>V(·)</code>	equity	how good the resulting position is	PORY	
the choice of <code>a</code>	strategy	a mixed strategy over <code>a</code> , not an argmax	SLOWKING	

The single most important structural fact is in the last row, and §6.2 is about it.

One term is genuinely finished, and it is the one poker does not get. `τ` is exact. Poker AIs must reason about a deck they cannot see; we own the generator — the open-source Showdown Champions engine — so chance is sampled exactly rather than modelled. This converts the transition function from the hardest thing to learn into a function call, and it is the discovery that makes the ReBeL program tractable here at all (§4c).

6.2 Mixing is not randomizing: you never fold aces preflop

A recurring misreading of equilibrium play is that it means *playing randomly so as to be unpredictable*. It does not, and the distinction decides whether this architecture plays well or badly.

An equilibrium strategy mixes **only among actions whose counterfactual values are close**, and plays **pure** wherever one action dominates. No equilibrium in any poker game folds aces preflop: the EV gap is enormous, so the mix collapses to a point. Regret matching produces this automatically — a strictly dominated action accumulates negative regret and receives probability ~ 0 . Balance is a property of the *close* decisions, and it costs nothing on the easy ones.

The failure mode this rules out is real and specific: an agent that randomizes uniformly over its candidate list in the name of unexploitability. That agent is indeed hard to read and it loses to everyone, because it Protects into nothing and clicks resisted moves at equilibrium frequency.

And this is precisely why the value function is load-bearing rather than a refinement. Regret matching mixes according to the counterfactual values it is handed. Hand it good values and it plays pure where the game is clear and balanced where it is close. Hand it a material-only evaluator and the "close" set is drawn wrongly — moves that are actually dominated look comparable, so the equilibrium mixes into them. **A Nash mix over wrong EVs is not a safe default; it is a worse player than greedy over the same wrong EVs, because greedy at least commits to the best of them.** This is the strongest available argument for sequencing PORY (§6.5) before search.

6.3 Unexploitable is not the same as winning

Exploitability is the project's scoreboard (§1, and the measured 63%), and it is the right scoreboard for the question *"can a prepared opponent read us?"* It is **not** the same question as *"do we win the most games against the field we actually face."*

- A Nash strategy guarantees you cannot **lose** much to anybody. It does not maximise winnings against **weak** opponents; it declines the exploits that beat them.
- A maximally exploitative strategy best-responds to the opponent's observed tendencies. It beats a weak field by more, and it is itself exploitable by a strong one.

Poker practice is to play exploitatively where reads are reliable and fall back toward equilibrium against strong or unknown opposition. The same choice is open here and has not yet been made deliberately: the public ladder is not an equilibrium field, so an agent optimised purely for low exploitability may leave a large amount of measurable win rate on the table against it. Both targets are legitimate; they are different objectives and should not be conflated in a single reported number.

6.4 Where the cost is — measured

The horizon is short and the branching is enormous (§4b). Measured over **7,254 real decisions** from 300 clean open-sheet games, using `board.js` 's own candidate builder:

level	size	cost at ~50 ms a rollout
one Pokémon choosing	mean 7.1 (median 7, p90 9, max 10)	trivial
my turn — slot A × slot B	~51	~3 s
the matrix — mine × theirs	~2,584	~2 minutes

This reconciles two claims in the project's own documents that appear to contradict each other. "Do not prune — recall is 87.9% at top-5 and the mean decision has 7.3 options" is a statement about the **per-slot** list, and it is correct: pruning 7 candidates to 5 discards 12% of the genuinely best moves to save four rollouts. "Breadth, not depth, is the binding constraint, and action abstraction is where the real work is" (§4b) is a statement about the **joint matrix**, and it is also correct: 2,584 cells is two minutes per turn.

They are about different levels, and the resolution is the design: **MAG orders the per-slot list and prunes nothing; abstraction happens on the joint action space**. The costing in the current plan — top 5 × their 3 likeliest × 3 samples ≈ 45 rollouts ≈ 2 s — is an action abstraction. It has simply never been named as one, and abstractions in poker are judged by a specific criterion: how much exploitability the abstraction itself introduces. That has not been measured here.

6.5 Status, term by term

term	status	evidence
T transition	done, exact	the real engine; 31 of 31 damage cases within 2% of <code>@smogon/calc</code>
<code>σ_{opp}</code> opponent frequencies	the best model in the project	XATU team context +0.0324 [0.021, 0.0435]
strategy / mixing	built, unit-tested	<code>nash.py</code> recovers RPS and an asymmetric 2×2 exactly
<code>P(r)</code> range	weak	CHOMP 51.3% [49.4, 53.2] — a coin, and it loses to "their four most-used"
V equity	the bottleneck	material-only. Richer features help and the network hurts (0.6154 logistic vs 0.6203 baseline)

Four of five terms are in hand. **EV cannot be computed until v is a value function rather than a material count** — and by §6.2, a search built on the current v would not merely be less accurate, it would mix into dominated actions and play worse than the greedy policy it replaces.

7. Training plan (the ReBeL upgrade and the data question)

Two coupled needs, one answer.

The leaf value network and policy network are trained by **self-play** on the real engine, exactly as ReBeL trains on poker self-play. Self-play also resolves ABRA's chronic **sample-size and selection-bias problem**: public ladder replays are a small, self-selected slice of games played (people save wins and flashy games), so any statistic learned from them is biased and scarce (~2,000 usable human games after dedup). **Engine self-play is unlimited and bias-free by construction** — it is simultaneously the data source for the value net and the cure for the data problem. The loop: (1) self-play with the current policy on the open engine; (2) label PBSs with re-solved values; (3) train value+policy nets; (4) repeat. Held-out human games remain the *calibration* set, scored with proper scoring rules (log-loss, Brier, reliability) — never the training signal, to avoid laundering the selection bias back in.

8. Honest limitations and open problems

- **Not yet a bot.** The solver is correct and tested; it is *waiting on* the engine adapter and the value net. We do not claim superhuman play today; we claim a sound chassis and a concrete path.
- **Action abstraction is unsolved here.** How aggressively to bucket moves/targets without discarding the equilibrium is real research.
- **Belief realism.** Our set priors are behavior-cloned from biased ladder data; the belief is only as good as those priors until self-play corrects them.
- **Equilibrium vs. exploitation.** Nash play is unexploitable but not maximally exploitative against a *known weak* opponent. Poker's exploitative extensions (e.g., modelling opponent deviations) are a natural follow-on, not a v1 feature.
- **Compute.** ReBeL-scale training is nontrivial; a scaled-down value net over abstracted states is the realistic first target.

9. Conclusion

VGC has, for years, been analyzed with damage calculators and usage statistics — the equivalent of poker "hand-strength charts." The lesson of the last decade of poker research is that the game is won by **belief-based, equilibrium search**, and that once the transition dynamics are available (here: the open engine), that machinery ports over cleanly, with three well-understood modifications for simultaneity, scale, and chance. SLOWKING is the attempt to bring that machinery to Pokémon in full, honestly, and with its evaluation held to the same standard poker researchers held themselves to.

References

- Zinkevich et al., *Regret Minimization in Games with Incomplete Information* (CFR), 2007.
- Moravčík et al., *DeepStack: Expert-Level AI in Heads-Up No-Limit Poker*, Science 2017.
- Brown & Sandholm, *Superhuman AI for heads-up no-limit poker: Libratus*, Science 2017.

- Brown et al., *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games (ReBeL)*, NeurIPS 2020.
- Schmid et al., *Player of Games*, 2021.
- *Hannan-consistent selection / SM-MCTS in simultaneous-move games*.
- Lanctot et al., *A Unified Game-Theoretic Approach to MARL (PSRO)*, NeurIPS 2017.
- Lopez, Matthews, Baumer, *How often does the best team win?* (skill vs. luck), 2018.